



HACKFEST 2026

Problem Statement Documentation & Technical Submission

Team ID	KH037
Team Name	GoldMiners
Team Leader Name	Sanket Mahajan
Problem Statement No.	NA
Problem Statement Name	VaaniMudra: An AI-Powered Speech-to-Indian Sign Language Translation System
Date of Submission	11/09/2026



SECTION 1 — TEAM DETAILS

Team ID – KH037

Team Name - GoldMiners

Team Leader Name – Sanket Mahajan

Team Members Names -

1. Onkar Sathe 2. Pruthviraj Gavhane 3. Sakshi Khutwad

Problem Statement Number - NA

Problem Statement Name

VaaniMudra - AI-Powered Speech-to-Indian Sign Language Translation Application with 3D Avatar Rendering

SECTION 2 — UNDERSTANDING OF THE PROBLEM

Problem Statement

There is no such live and end-to-end system which will convert English speech into animation of ISL. In India, by the year 2050, there will be 2.7 million DHH persons along with 5 million plus people using ISL, and lack of translation technology will result in a big communications gap between DHH persons and the rest of the world.

This gap will be bridged by VaaniMudra in the form of a mobile application translating English speech or text into ISL through animation of virtual avatar by implementing an AI-based natural language processing pipeline to understand the sense of the sentence and translate it into ISL grammatical format (GLOSS).

Problem Analysis / Background

In DHH community of India, the language spoken is Indian Sign Language (ISL) that has SOV structure, spatial grammar, non-manual features like facial expressions and head movements, and does not contain articles and auxiliaries. This makes the English to ISL translation linguistically invalid.

Existing problems that will be addressed:

- There are 3,000-7,000 ISL machine readable signs, which will create severe out-of-vocabulary problems that will have to be addressed through semantic approximation.
- The current technologies for ISL will be fragmented; working independently with speech recognition, sign dictionary or avatar generation but not collectively in the pipeline process.
- The rule-based system has limitations, whereas deep learning-based gesture recognition will require huge volumes of labeled data in ISL that are not available yet.
- Majority of hearing individuals will not understand ISL, creating significant communication challenges.

Who is affected: Deaf and hard-of-hearing individuals, their families, teachers, health care professionals, and anyone else who may need to interact with ISL speakers.

Proposed Solution

VaaniMudra is the project that intends to create an Artificial Intelligence-enabled software that will translate from spoken/written English language into animated Indian Sign Language using a 3D avatar on Android and iOS platforms.

How it will work:

1. The user speaks/writes/utters the sentence in English ("I am going to school")
2. The AI Engine deciphers the grammar of the sentence, extracts the meaning of the sentence and converts it into ISL GLOSS ("[SCHOOL] [GO]")
3. An animated character makes the signs of ISL in real time on the user's screen.
4. When a single word is being looked up, the Word Mode will provide results instantly.

Key features:

- Speech input to translate into ISL using multilingual features (en-US, en-IN, en-GB)
- Combination of an NLP model (spaCy) and a language model (LLM) for meaning extraction (NVIDIA GPT).
- Real-time generation of 3D avatars using CWASA WebGL engine.
- Two methods are included: offline Word Mode (JSON search) and online Sentence Mode (AI translation).
- Use of semantic search through embeddings in case unknown words appear.
- Cross-platform application development using React Native framework.

SECTION 3 — RESEARCH

Research Conducted

The research on sign language translation technology was done in the following fields:

- **HamNoSys and SiGML:** Hamburg Notation System (HamNoSys), which is a phonetic notation of sign languages, and SiGML, which is an XML encoding of sign gestures. The sign animation that would be employed in this research would be encoded using SiGML.
- **CWASA Avatar System:** CWASA stands for the Client-side WebGL Avatar for Sign Animation developed by the University of East Anglia. It was analyzed and integrated in the app via a WebView to render the 3D sign animations.
- **Semantic embedding-based retrieval:** Sentence-BERT and transformer embeddings as well as ChromaDB vector database would be used for handling out-of-vocabulary words.
- **ISL linguistics:** Linguistic research on ISL showed that SOV grammatical construction implies grammar transformation module to be added to the English-to-ISL pipeline.
- **Sign language data:** ISLRTC dictionary and other repositories of ISL language created by the ISL community members would be the sign language data.

Existing Solutions / Similar Work

Current sign language translation systems are generally focused on ASL or European sign languages while systems for ISL translation have not been highly developed enough to be practical.

Existing approaches:

- Models based on rules: inflexible, incapable of generalizing beyond programming, and require manual input of linguistic knowledge.
- Models based on deep learning for gesture recognition: depend on big data sets, which are not available for ISL.
- Models based on avatar animations: can only be used experimentally and are not included in speech-to-sign pipelines.

How SignVision will be different or improved:

- This will be the first ever GLOSS generation software for ISL which uses LLM techniques and deterministic ISL grammar.
- A multilevel hierarchical lookup of words (exact, anchor, partial, semantic fallback) will be used to improve word recognition and to handle unknown words.
- The entire pipeline, from speech input to a rendered 3D animated avatar, will be available within a single cross platform mobile app.
- The offline capable "Word Mode" will have a pre-installed JSON data set.

References / Sources

1. N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in Proceedings of EMNLP, pp. 3982-3992, 2019. <https://aclanthology.org/D19-1410/>
2. S. Kadashetti, P. Bhoyar, and D. Jawar, "Review on Indian Sign Language recognition with machine learning," International Journal of Engineering Applied Sciences and Technology, vol. 7, no. 1, pp. 68-74, 2022.
3. M. Kumari and M. Goyal, "Translation of Indian Sign Language with deep learning: A review," Journal of Physics: Conference Series, vol. 1950, 2021.
4. University of East Anglia Virtual Human Group, "CWASA: Client-side WebGL Avatar for Sign Animation," 2020. <https://vhg.cmp.uea.ac.uk/tech/jas/vhg2020/>

SECTION 4 — TECHNICAL DOCUMENTATION

Technology Stack

Frontend:

- React Native (Expo 55) using TypeScript for developing an all-encompassing multi-platform mobile application.
- Using expo-speech-recognition to turn speech into text locally.
- react-native-webview to incorporate the CWASA avatar on the app.

Backend & AI:

- FastAPI using Python for the AI backend and REST API.
- spaCy (en_core_web_trf - based on RoBERTa transformers) for POS tagging and dependency parsing.
- NVIDIA GPT API (openai/gpt-oss-120b, 120B parameters) for semantic extraction.
- SentenceTransformer (all-MiniLM-L6-v2, 384-dimensional) for semantic embeddings.
- Chroma

Data & Cloud:

- Amazon S3 to host animation files [.sigml].
- SiGML (HamNoSys notation) to represent the sign gesture encoding standard.

Working / Implementation Approach

The system will follow a modular multi-stage pipeline:

Stage 1 - Speech Recognition:

expo-speech-recognition (mobile) or Google Speech Recognition API (backend) will transcribe speech to text, returning interim results for immediate visual feedback, as well as supporting multiple locales (en-US, en-IN, en-GB).

Stage 2 - Text Preprocessing:

The text will be lower case, contractions expanded (about 60 entries curated), sentences tokenized with punctuation marks, and punctuation/whitespace normalized.

Stage 3 - Linguistic Analysis:

spaCy en_core_web_trf (transformer based on RoBERTa) will provide reliable POS tagging and dependency parsing.

Stage 4 - LLM Semantic Extraction:

The text and its annotations from spaCy will be fed into the NVIDIA GPT API (openai/gpt-oss-120b), which will provide the following information in a structured JSON format: subject, object, normalized verb, tense (PAST/PRESENT/FUTURE), sentence type, negation flag, predicative adjective, and confidence score. Temperature 0.1 will be used to obtain deterministic output, while there will be retries below 95 confidence and a spaCy-only fallback.

Step 5 - ISL Rule Engine and GLOSS Token Generation:

A rule engine will convert the semantic fields into GLOSS tokens according to ISL rules: article deletion, aux deletion, time markers addition (YESTERDAY/FUTURE/NOW), wh-words reposition to sentence end, NOT addition (negation), Q addition (yes/no questions). Output order: [TIME] [SUBJECT] [OBJECT] [VERB] [ADJECTIVE] [NOT] [WH-WORDS].

Step 6 - Word Lookup:

A four-step word lookup algorithm will be implemented for each GLOSS token: exact match, anchor/synonym match, substitution match, and semantic match by embedding using SentenceTransformer + ChromaDB cosine similarity (threshold of 0.5).

Step 7 - 3D Avatar Sign Rendering:

Signing will be performed by CWASA WebGL 3D avatar on-the-fly. Communication between React Native application and the avatar will occur via postMessage API (play requests and play finished statuses).

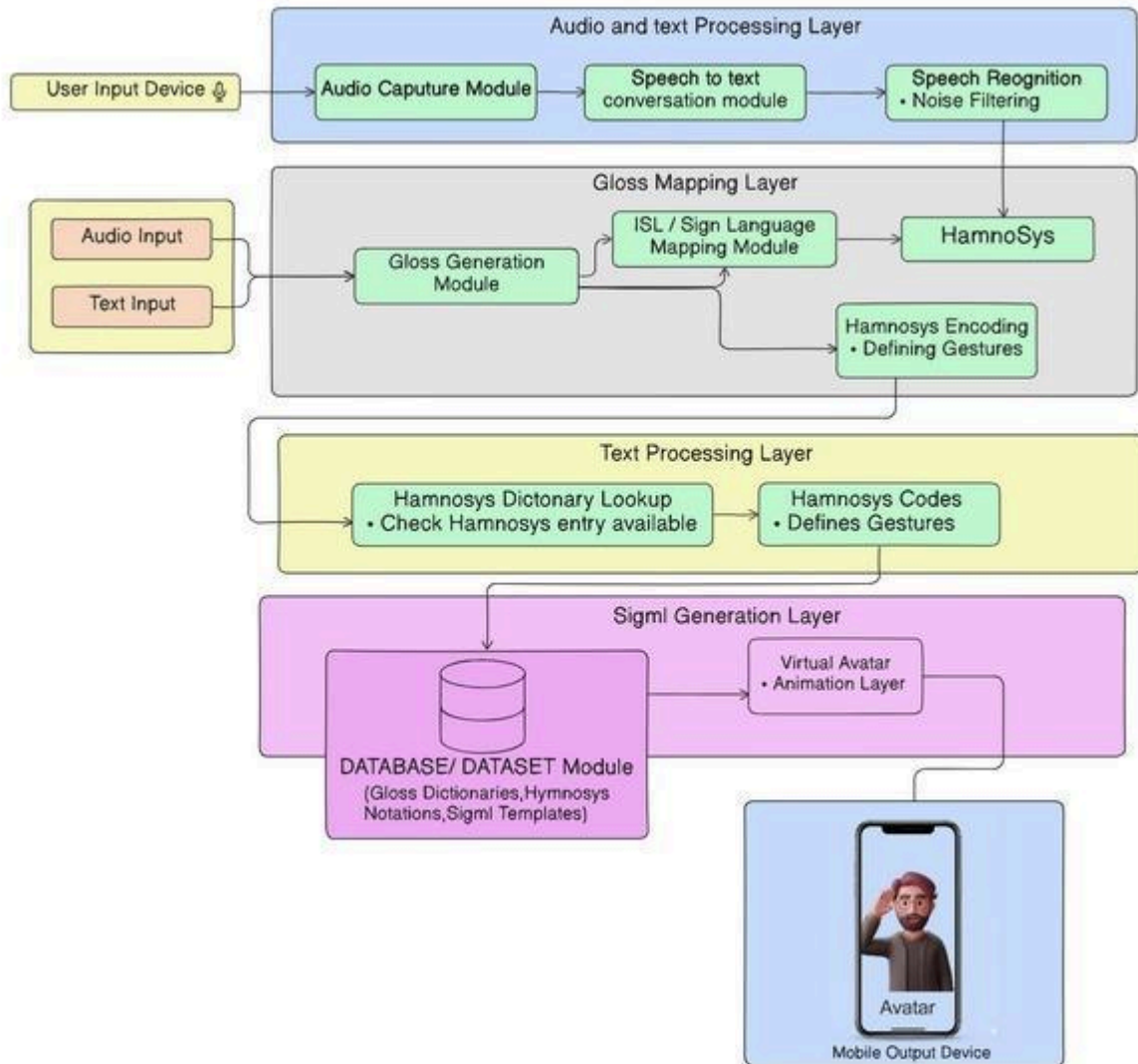


Figure: System Architecture Diagram

Technical Flow / System Architecture Diagram

Flowchart for system architecture:

1. [Mobile App] User makes voice or text input into SearchBar component.
2. [Mobile App] Word Mode uses built-in local JSON file for signing dictionary.
3. [Mobile App] Sentence Mode sends text to the backend through REST API (POST /process) with retry and timeout handling mechanism.
4. [Backend] Text preprocessing: lowercase, contraction expansion, sentence splitting, punctuation removal.
5. [Backend] spaCy en_core_web_trf does linguistic processing (part-of-speech tagging, dependency parsing).
6. [Backend] NVIDIA GPT-120B extracts semantic features (subject, object, verb, tense, negation, sentence type).
7. [Backend] ISL rule engine uses ISL grammar to generate GLOSS tokens in SOV order.

-
8. [Backend] Four-tier word lookup translates GLOSS tokens into SiGML URLs, if not found - ChromaDB Semantic Search is used as a fallback solution.
 9. [AWS S3] SiGML animation files are fetched via HTTPS protocol.
 10. [Mobile App] CWASA WebGL Avatar loads each .sigml file sequentially with postMessage bridge via WebView.

AI/ML Details

Dataset:

A lexical database of ISL created from the ISLRTC dictionary and crowd-sourced sign databases, alphabetically sorted in the dataset/directory (~1,500+ words in SiGML format and ~20,000 words that were used for creating word vectors).

Models / APIs used:

- spaCy en_core_web_trf (transformer-based on RoBERTa) – POS tagging and linguistic processing.
- NVIDIA GPT API (openai/gpt-oss-120b) – semantic extraction based on an LLM through an OpenAI-compatible endpoint.
- SentenceTransformer all-MiniLM-L6-v2 – semantic embeddings with 384 dimensions.
- ChromaDB (HNSW algorithm, cosine similarity) - vector search for out-of-vocabulary word resolution.

Approach:

Pre-computation of word embeddings is carried out only once per full word list and stored in ChromaDB. During execution, the LLM uses zero/low temperature for getting semantic fields, while the retry mechanism with the help of confidence ensures reliability of all extractions, which will be done with the help of ISL rule engine, generating GLOSS tokens.

GitHub Repository Link

<https://github.com/GoldMiners141/vaanimudra>

SECTION 5 — ORIGINALITY & PLAGIARISM DECLARATION

- ✓ I confirm that the solution submitted by our team was developed during the hackathon. We have not submitted a pre-built project as our primary solution. Any open-source code, APIs, datasets, libraries, templates, or AI-assisted development used by us have been appropriately acknowledged.

Open-Source / External Resources

- CWASA Avatar Library (University of East Anglia) – open source WebGL sign language avatar library.
- spaCy (en_core_web_trf) – open source NLP library and transformer.
- SentenceTransformers (all-MiniLM-L6-v2) – open source embedding model from Hugging Face.
- ChromaDB – open source embedded vector database.
- React Native / Expo – open source mobile app development environment.
- ISLRTC Dictionary – ISL dictionary by the Government.
- AWS S3 – cloud storage of [.sigml] animation files.

AI Tools

AI tools were used at various stages of development:

NVIDIA GPT API (openai/gpt-oss-120b):

- used for the LLM that extracts the sentence structure semantics of English sentences when ISL GLOSS is being generated by the backend NLP system.

OpenAI-compatible SDK:

- used for communicating with the LLM API endpoint.

SentenceTransformer embedding model:

- used for creating the word embeddings for the semantic search feature.

AI development assistance software:

- used for assistance in coding, research, and documentation.



SECTION 6 — UPLOADS / ATTACHMENTS CHECKLIST

- ✓ Technical Flow / Architecture Diagram
 - ✓ Project Documentation / Technical Report
 - ✓ GitHub Repository Link
-

Team Leader Signature: S. S. Mahajan

Date: 11-09-2026